Deep Learning and Its Applications (CO3133) – Assignment 1

Foundations of Deep Learning Pipelines and Architectures

From Linear Models to Modern Sequence Models: A Comparative Study for Image Classification

Le Pham Tien Long[1,2(✉)] Ngo Tieu Nghi[1,2] Ho Minh Nhat[1,2]

[1] Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), 268 Ly Thuong Kiet, Dien Hong Ward, Ho Chi Minh City, Vietnam.
[2] Vietnam National University Ho Chi Minh City (VNU-HCM), Linh Xuan Ward, Ho Chi Minh City, Vietnam.
Instructor: Le Thanh Sach **Semester:** 261 **Group:** 18 – DeepDive
{long.lephamtien, nghi.ngotieu, nhat.hominh}@hcmut.edu.vn

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. Conclusion

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. Conclusion

Problem Formulation

- **Task:** single-label image classification, one prediction per image.
- **Input:** a grayscale image $x \in \mathbb{R}^{1\times 28\times 28}$.
- **Output:** raw logits $z = f_\theta(x) \in \mathbb{R}^{10}$, one score per class; the prediction is $\hat{y} = \arg\max_c z_c$.
- **Research question:** with the data, the split and the training protocol fixed, how does the model family, and the way it reads the image, change macro-F1, accuracy and cost?
- **Datasets:**
  - Fashion-MNIST [1] for every reported result.
  - MNIST for debugging the pipeline only; no MNIST number is reported.
- **Main difficulty:** four upper-body classes share one silhouette at 28×28 pixels and without colour.

Fashion-MNIST

- **Source:** Zalando product photos, converted to grayscale and resized [1].
- **Size:** 70,000 images; 60,000 official training, 10,000 official test.
- **Format:** 28×28, one channel, `uint8` in $[0, 255]$; about 30 MB compressed.
- **Classes:** T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.
- **License:** MIT.
- **Balance:** every class holds exactly 10% of each split (Figure 1), so there is no class imbalance to correct.

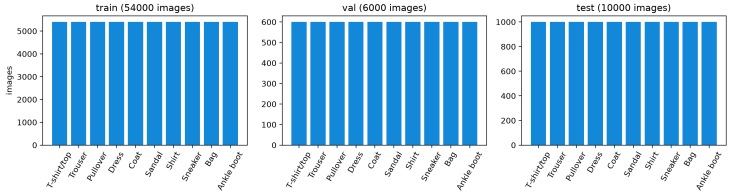


Figure 1: Images per class in the train, validation and test splits.

Representative Samples

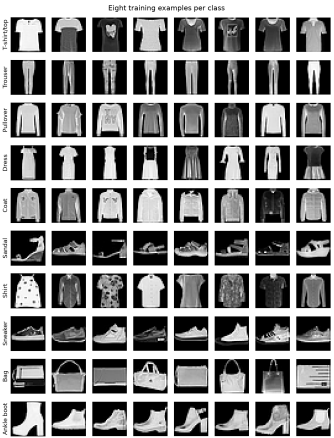


Figure 2: Eight training images per class.

- **Distinct shapes:** Trouser, Bag and the three footwear classes have outlines no other class shares (Figure 2).
- **Shared silhouette:** T-shirt/top, Pullover, Coat and Shirt differ mainly in sleeve length, collar, buttons or zip, details that cover only a few pixels.
- **Variation within a class:** prints, brightness and cut change from image to image.
- **Fixed framing:** every garment is centred on a black background.

Class Overlap Before Training

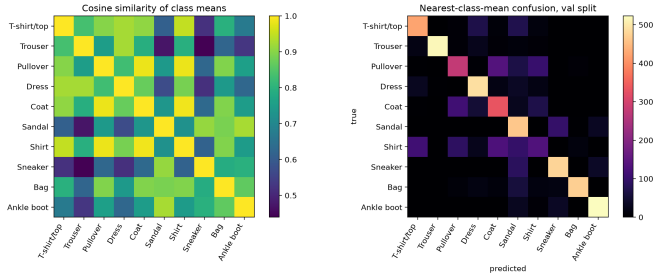
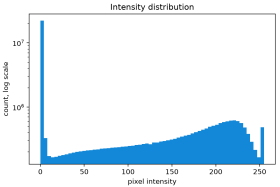


Figure 3: Left: cosine similarity between class mean images. Right: confusion matrix of a nearest-class-mean classifier on the validation split.

- **Similar means:** cosine similarity exceeds 0.98 for Coat/Shirt, Pullover/Coat and Pullover/Shirt.
- **Floor without training:** assigning each image to the closest class mean reaches 68.4% validation accuracy (Figure 3).
- **Uneven classes:** Shirt recall is 0.220, against 0.872 for Ankle boot.

Pixel Statistics and Spatial Structure



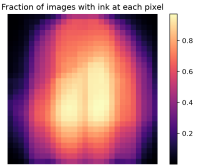


Figure 4: Pixel intensity histogram; share of images with a non-zero value at each pixel.

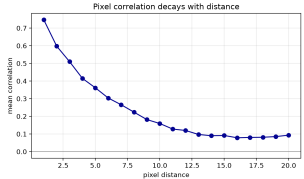


Figure 5: Mean pixel correlation against distance.

- **Sparse input:** half of all training pixels are exactly 0, and the garment occupies the centre (Figure 4), so many of the 784 flattened features carry little information.
- **Redundancy:** 10 principal components explain 72.1% of pixel variance; 100 explain 91.4%.
- **Local structure:** pixel correlation falls from 0.747 at 1 pixel to 0.159 at 10 and 0.093 at 20 (Figure 5). A convolution uses this structure; flattening removes it.

Three Sequence Views of One Image

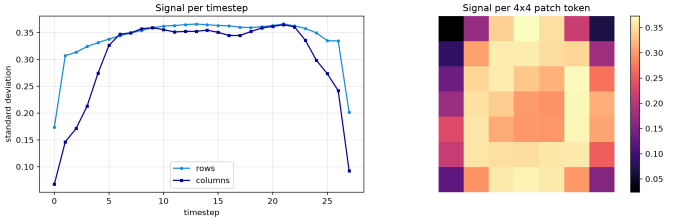


Figure 6: Standard deviation of pixel values per timestep for row and column sequences (left) and per 4×4 patch token (right).

- **Rows or columns:** 28 timesteps of 28 features. **Patches of 4×4:** 49 tokens of 16 features.
- **Where the signal sits:** no row falls below a quarter of the peak standard deviation, against one column and three patch tokens (Figure 6). Patches gather the background into a few corner tokens; rows and columns spread it over every step.

What the Data Implies for the Method

- **Balanced classes:** accuracy and macro-F1 are both valid; macro-F1 is primary because it weights a weak class such as Shirt as much as a strong one.
- **Upper-body overlap:** the error analysis should focus on T-shirt/top, Pullover, Coat and Shirt, and the nearest-class-mean result (68.4%) is the floor every trained model must clear.
- **Local correlation:** a model with a spatial prior, the CNN, is expected to lead at comparable capacity (experiment E1).
- **Uneven signal per token:** patch tokens are expected to beat row and column tokens for the recurrent and Transformer models (experiment E2).
- **Centred garments with small shifts:** random crops of ± 2 pixels and horizontal flips produce plausible training images without changing the label.

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. Conclusion

Pipeline Overview

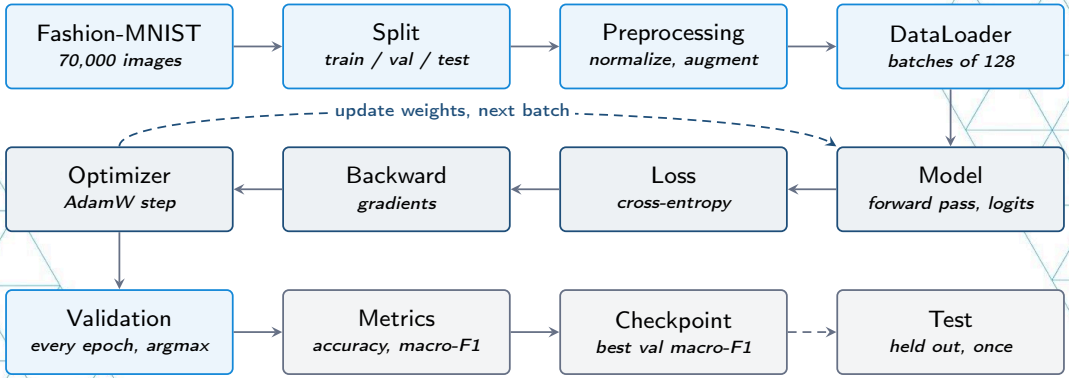


Figure 7: From the dataset to the selected checkpoint; the same pipeline trains all five models.

- **Shared path:** only the *Model* box changes between experiments (Figure 7).
- **Held-out test:** the test split is evaluated once, after model selection on validation macro-F1.

Preprocessing and Batches

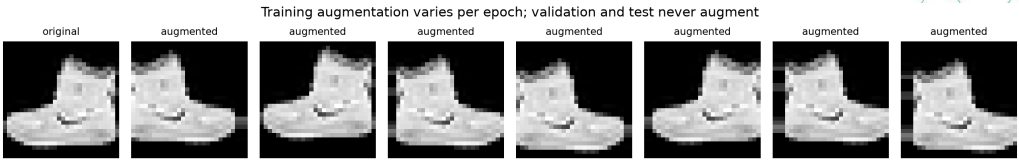


Figure 8: One training image and seven augmented versions of it.

- **Normalization:** $\tilde{x} = (x/255 - 0.2858)/0.3529$, with the mean and standard deviation computed on the training split only.
- **Augmentation, training loader only:** random crop with 2-pixel edge padding, then horizontal flip with probability 0.5 (Figure 8). Validation and test images are never augmented.
- **Batch:** images $(128, 1, 28, 28)$ `float32`, values in $[-0.81, 2.02]$; labels $(128,)$ `int64`. Every model receives this same batch.

Shared Design of the Five Models

- **One output contract:** every model maps a batch to raw logits of shape $(N, 10)$. No model applies softmax; the loss does it:

$$\mathcal{L}(z, y) = -\log \frac{\exp(z_y)}{\sum_{c=1}^{10} \exp(z_c)}$$

- **Registry:** models are registered by name and built from a YAML configuration, so the training loop has no model-specific code.
- **Sequence input:** the recurrent and Transformer models receive the same image batch and convert it into rows, columns or patches themselves.
- **Self-implemented:** the five architectures, the data split, the training and evaluation loops, and run tracking.
- **From libraries:** PyTorch [2] layers (`Linear`, `Conv2d`, `GRU`, `LSTM`, `TransformerEncoderLayer`), optimizer and scheduler; torchvision for download and transforms; scikit-learn for the stratified split and the metrics. No pretrained weights are used.

Model 1: Linear Classifier (Baseline)

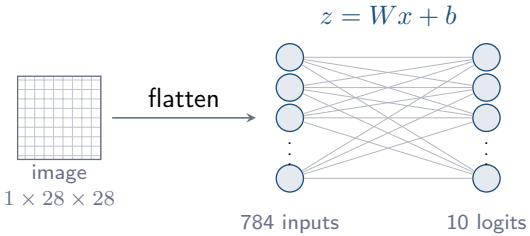


Figure 9: The linear classifier: one affine layer from flattened pixels to class scores.

- **Computation:** $x \in \mathbb{R}^{784}$, $W \in \mathbb{R}^{10\times784}$, $b \in \mathbb{R}^{10}$ (Figure 9); 7,850 parameters.
- **What it can represent:** each class score is a weighted sum of pixels, so the decision boundaries between classes are hyperplanes in pixel space.
- **Role:** the capacity floor of the comparison; every other model must improve on it.

Model 2: Multilayer Perceptron (MLP)

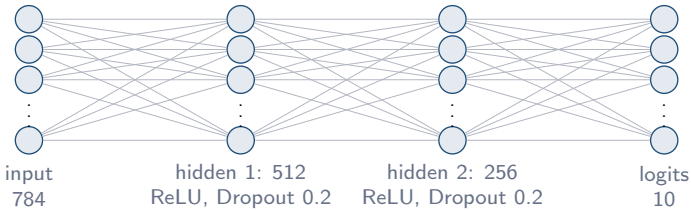


Figure 10: The MLP: two fully connected hidden layers between the flattened image and the logits.

- **Computation:** $h_1 = \mathrm{Dropout}(\mathrm{ReLU}(W_1 x + b_1))$, $h_2 = \mathrm{Dropout}(\mathrm{ReLU}(W_2 h_1 + b_2))$, $z = W_3 h_2 + b_3$ (Figure 10); 535,818 parameters.
- **Activation:** $\mathrm{ReLU}(a) = \max(0, a)$ makes the model non-linear, and its gradient does not shrink for positive inputs.
- **Regularization:** dropout [3] sets 20% of hidden units to zero during training; weight decay 0.05 comes from the shared optimizer.
- **Limit:** like the linear model, it reads a flat vector and has no notion of which pixels are neighbours.

Model 3: Convolutional Neural Network (CNN)

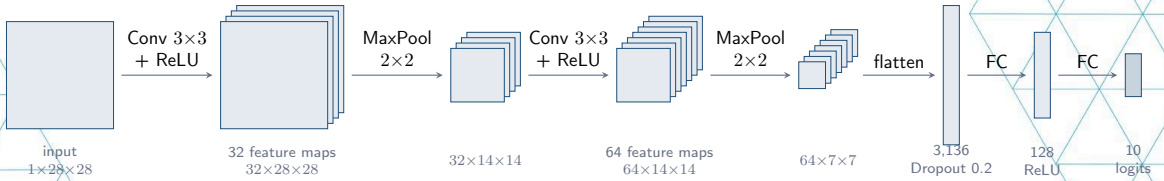


Figure 11: The CNN: two convolution stages, then a fully connected head; 421,642 parameters.

- **Convolution:** each 3×3 filter is applied at every position with the same weights; padding 1 keeps the 28×28 size. One filter produces one feature map, which records where that filter responds (Figure 11).
- **Pooling:** 2×2 max pooling keeps the largest response in each window. It halves the map size and makes the output less sensitive to shifts of one pixel.
- **Design:** our own architecture after LeNet [4], two stages of 32 and 64 filters, trained from scratch.

Model 4: Recurrent Network (GRU)

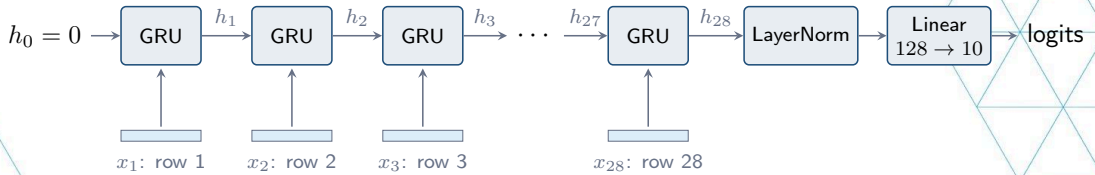


Figure 12: The recurrent classifier reads the image one row per timestep; 62,218 parameters with a GRU.

- **Timestep and input size:** one timestep is one image row, read top to bottom, so the sequence has 28 steps of 28 features (Figure 12). Columns and 4×4 patches are the alternatives tested in E2.
- **Hidden representation:** $h_t = \mathrm{GRU}(x_t, h_{t-1}) \in \mathbb{R}^{128}$ summarizes the rows read so far; the last state h_{28} is classified.
- **Gates:** the GRU [5] uses an update and a reset gate to decide how much of h_{t-1} to keep. The LSTM [6] adds a separate cell state and a third gate (82,442 parameters); E3 compares the two.

Model 5: Transformer Encoder

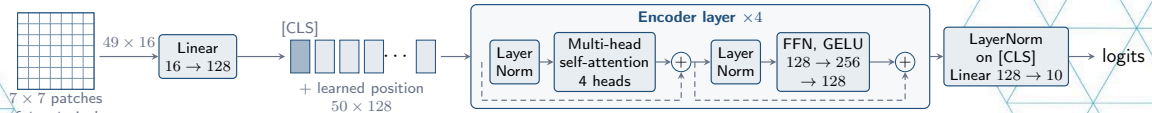


Figure 13: The Transformer classifier over 4×4 patch tokens; 540,170 parameters.

- **Tokens:** each patch is flattened to 16 values and projected to $d = 128$; a learned [CLS] token is prepended, and its final state is classified (Figure 13), as in ViT [7].
- **Positional encoding:** attention ignores token order, so one learned vector per position is added to each token.
- **Attention [8]:** queries, keys and values are linear projections of the same tokens H; each of the 4 heads computes $\mathrm{softmax}(QK^\top/\sqrt{32})\,V$, so every token can use information from every other token in one layer. Input and output are both $(N, 50, 128)$.

The Five Models Side by Side

Table 1: Input view, built-in assumption and size of each model, at the configurations in `configs/`.

Model	Input view	Built-in assumption (inductive bias)	Parameters
Linear	784 pixels	class scores are linear in the pixels	7,850
MLP	784 pixels	none about pixel layout	535,818
CNN	28×28 grid	nearby pixels matter together; same filter everywhere	421,642
GRU	28 rows of 28	the image is an ordered sequence of rows	62,218
Transformer	49 patches of 16	few; positions are learned, all tokens interact	540,170

- **No spatial prior:** Linear and MLP would learn equally well if every image had its pixels shuffled by the same fixed permutation; the CNN and the patch Transformer would not.
- **Unequal size:** parameter counts differ by a factor of 69 (Table 1), so results are read together with cost, and E4 compares the models at a matched parameter budget.

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. Conclusion

Data Split and Leakage Control

Table 2: The split shared by every model, stratified by class with seed 0.

Subset	Images	Per class	Source
Train	54,000	5,400	official training set, 90%
Validation	6,000	600	official training set, 10%
Test	10,000	1,000	official test set

- **Fixed partition:** the split indices are committed in `splits/fashion-mnist-seed0.json` and read by every run (Table 2), instead of being regenerated from a seed that depends on library versions.
- **No information from validation or test in training:** normalization statistics come from the training split only, and augmentation is applied to the training loader only.
- **Single test evaluation:** checkpoints and configurations are chosen on validation; the test split is evaluated once, after model selection concludes.

Training Protocol

Table 3: Training settings shared by all five models.

Setting	Value
Loss	cross-entropy on raw logits, no label smoothing
Optimizer	AdamW [9], learning rate 3×10^{-4}, weight decay 0.05
Batch size	128
Schedule	cosine annealing [10] over 30 epochs
Early stopping	after 5 epochs without improvement in validation macro-F1
Checkpoint	epoch with the highest validation macro-F1
Augmentation	random crop with 2-pixel edge padding, horizontal flip
Seeds	0, 1, 2 for every reported configuration

- **Environment:** Apple M4 Pro (14 cores, 48 GB), PyTorch MPS backend; Python 3.12.0, PyTorch 2.14.0, torchvision 0.29.0, scikit-learn 1.9.1.
- **Run record:** each run stores its resolved configuration, commit hash, library versions and hardware, so every row in the results can be traced to the settings in Table 3.

Metrics

- **Macro-F1 (primary):** the mean of the per-class F1 scores,

$$\mathrm{F1}_c = \frac{2\,P_c R_c}{P_c + R_c}, \qquad \text{Macro-F1} = \frac{1}{10}\sum_{c=1}^{10} \mathrm{F1}_c,$$

  where P_c and R_c are the precision and recall of class c. A weak class lowers the score even when overall accuracy is high.

- **Accuracy:** the share of correctly classified images.

- **Cost:**
  - number of trainable parameters;
  - training time, in seconds per epoch and in total;
  - inference latency in milliseconds per image at batch sizes 1 and 128, averaged over 50 timed batches after 5 warm-up batches.

- **Qualitative evidence:** learning curves, the confusion matrix, and correct and incorrect prediction examples for each run.

Seeds, Tuning Budget and Decision Rule

- **Repeated runs:** every reported configuration is trained with seeds 0, 1 and 2 and reported as mean $\pm$ one standard deviation.
- **Equal tuning budget:** at most six configurations per model, compared on validation macro-F1 at seed 0; only the model's own arguments change.
- **Decision rule:** model A is better than model B only if its mean validation macro-F1 is higher *and* the two intervals of mean $\pm$ one standard deviation do not overlap (Figure 14). Otherwise the two are reported as indistinguishable.
- **Accuracy alone never decides a comparison.**

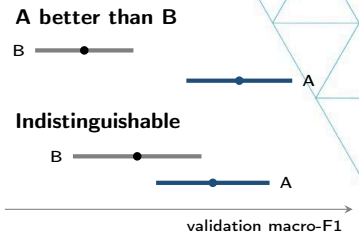


Figure 14: Mean $\pm$ one standard deviation over three seeds.

Experiments and Hypotheses (1/2)

E1: Architecture families

Hypothesis: models with a spatial prior outperform models that read the image as an unordered vector, at comparable capacity.

Varies: model family: Linear, MLP, CNN, GRU, Transformer.

Fixed: the training protocol and the tuning budget.

Metrics: validation macro-F1, accuracy, parameters, seconds per epoch, latency at batch 1 and 128.

E2: Sequence representation

Hypothesis: patch tokens outperform row and column tokens for the recurrent and Transformer models, because a patch keeps neighbouring pixels in both directions together.

Varies: rows, columns, or 4×4 patches.

Fixed: the architecture, its arguments and the protocol.

Metrics: validation macro-F1, against sequence length (28 or 49) and features per token (28 or 16).

Experiments and Hypotheses (2/2)

E3: Recurrent cell

Hypothesis: the GRU matches the LSTM with fewer parameters, because sequences of 28 or 49 steps are short enough that the LSTM's extra gate and cell state add little.

Varies: cell type, GRU (62,218 parameters on rows) or LSTM (82,442).

Fixed: representation, hidden size, depth, dropout and the protocol.

Metrics: validation macro-F1, parameters, seconds per epoch. If the two are indistinguishable, the smaller model is preferred.

E4: Matched budget (extension)

Hypothesis: the E1 ranking reflects the architecture rather than the parameter count, so it holds when all models have about the same number of parameters.

Varies: model family, with width tuned to within 10% of a shared budget.

Fixed: the budget and the protocol.

Metrics: validation macro-F1, with the realised parameter counts reported. E4 adds to E1 and does not replace it.

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. **Results**
5. Error Analysis
6. Conclusion

Results

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. Conclusion

Error Analysis

Table of Contents

1. Problem and Data
2. Method
3. Experimental Setup
4. Results
5. Error Analysis
6. Conclusion

Conclusion

Thank You

References I

[1] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.

[2] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.

[3] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.

[4] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[5] K. Cho et al., "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 2014, pp. 1724–1734.

References II

[6] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021.

[8] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.

[9] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[10] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *International Conference on Learning Representations*, 2017.